

Traditional-DIC

操作使用手册

中文版

Generated from the project documentation

目录

Traditional-DIC 使用说明手册

目录

项目架构

功能范围

跨平台编译

配置与案例目录

examples 完整求解流程

新手操作清单

配置参数详解

可复现实验流程

应变与结果

Python 接口

常见问题

Traditional-DIC 使用说明手册

目录

- [项目架构](#项目架构)
- [功能范围](#功能范围)
- [跨平台编译](#跨平台编译)
- [配置与案例目录](#配置与案例目录)
- [examples 完整求解流程](#examples-完整求解流程)
- [应变与结果](#应变与结果)
- [Python 接口](#python-接口)
- [新手操作清单](#新手操作清单)
- [配置参数详解](#配置参数详解)
- [可复现实验流程](#可复现实验流程)
- [常见问题](#常见问题)

项目架构

Traditional-DIC 由 C++17 数值核心、pybind11 扩展和 Python 工作流组成：

```
■■/ROI -> ■■■■ B-spline -> ■■■■  
      -> Subset-DIC ■ Mesh-DIC -> ■■■■  
      -> ■■/■■ -> ■■■■■■■■ -> ■■■■■■■■■■
```

include/dic/ 是 C++ 公共接口，src/ 是实现；bindings/python/ 编译 _traditional_dic；python/traditional_dic/ 是 Python API；examples/ 是正式案例程序。图像、掩膜、插值、相关准则和初始化是共享模块；Subset 管理圆形子区、种子和传播，Mesh 管理网格、节点初始化和全局有限元求解，三维模块只消费二维相关场。

功能范围

- Subset-DIC：ROI 截断圆形子区、镜像填充、整数/SIFT 初始化、种子选择、可靠性传播、一二阶形函数、SSD/ZNSSD、ICGN/FGN。
- Mesh-DIC：T3/Q4/Q8、ROI 自动网格或手工网格、FE-DIC FFT 初始化、稀疏全局求解、正则化、节点/稠密位移场。
- 3D-DIC：棋盘格标定、双目三角化、多目自标定、尺度恢复、相机对选择、两两重建和曲面拼接。

跨平台编译

必需：CMake 3.16+、C++17 编译器、Eigen3、yaml-cpp、Python 3.9+、NumPy、Pillow、PyYAML。按功能可选 OpenCV、Ceres、pybind11、SciPy、Matplotlib。examples 依赖 Python 绑定，必须以 TRADITIONAL_DIC_BUILD_PYTHON=ON 构建。

Windows + Visual Studio + Conda

- 安装 Visual Studio/Build Tools 的 C++ 桌面开发组件和 Windows SDK。
- 激活包含 Python、pybind11、Eigen、yaml-cpp、OpenCV 的 Conda 环境。
- 配置、编译并验证：

```
conda activate <environment>  
cmd /c 'call "C:\Program Files\Microsoft Visual Studio\18\Community\Common7\Tools\VsDevCmd.bat" -arch=x64 -host_arch=x64'  
cmd /c 'call "C:\Program Files\Microsoft Visual Studio\18\Community\Common7\Tools\VsDevCmd.bat" -arch=x64 -host_arch=x64'
```

```
python -c "import sys; sys.path.insert(0, 'python'); import traditional_dic; print('OK')"
```

Linux

安装 build-essential cmake libeigen3-dev libyaml-cpp-dev python3-dev, 创建虚拟环境并安装 numpy pillow pyyaml pybind11 scipy matplotlib:

```
python -m venv .venv && source .venv/bin/activate
pip install numpy pillow pyyaml pybind11 scipy matplotlib
cmake -S . -B build -DTRADITIONAL_DIC_BUILD_PYTHON=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
PYTHONPATH=python python -c "import traditional_dic; print('OK')"
```

macOS

安装 Xcode Command Line Tools、Homebrew 的 cmake eigen yaml-cpp, 然后创建 Python 虚拟环境、安装上述 Python 依赖, 并执行与 Linux 相同的 CMake 命令。所有平台必须保证 Python、OpenCV 和编译器架构一致; 找不到包时激活正确环境, 或设置 CMAKE_PREFIX_PATH、Eigen3_DIR、yaml-cpp_DIR、OpenCV_DIR。

配置与案例目录

算法 YAML 仅保存参数: subset_2d.yaml、mesh_2d.yaml、calibration.yaml、stereo_3d.yaml、multiview_3d.yaml。所有输入输出路径只写在 config/case_paths.yaml。

- 单目: images_dir 排序后的首图是参考、末图是 ROI、所有中间图是变形图。
- 双目: 左右目录首图参考、末图变形; 棋盘格图像目录独立设置。
- 多目 roi.mode: auto: 每个相机首图参考、末图变形, 自动生成 pair ROI。
- 多目 roi.mode: last_image: 首图参考、倒数第二张变形、末图为该相机 ROI; 该相机作为主相机时, ROI 用于时序与视差相关。

output.result_root、output.visualization_root 默认是 result、visualization。不要把案例路径写回算法 YAML。

examples 完整求解流程

单目 Subset

```
python examples\subset_2d.py --paths-config config\case_paths.yaml --config config\subset_2d.yaml
```

可覆盖 --radius --spacing --search-radius --seed-count --max-iterations。每个变形帧输出 result/subset/<frame>/displacements.csv、stats.json、strain.csv 和对应可视化。

单目 Mesh

```
python examples\mesh_2d.py --paths-config config\case_paths.yaml --config config\mesh_2d.yaml --element all
```

--element 为 T3|Q4|Q8|all。可覆盖初始化、优化器、目标函数、正则化、稠密采样和质量控制参数。每帧每单元输出网格、final_U.csv、dense_U.csv、strain.csv、summary 和图像。

双目 3D

```
python examples\stereo_3d.py --paths-config config\case_paths.yaml --stereo-config config\stereo_3d.yaml --calibration
```

完整流程为标定, 计算 $L0 \rightarrow R0$ 参考视差、 $L0 \rightarrow L1$ 时序场、 $L0 \rightarrow R1$ 变形视差, 三角化得到 $X0$ 、 $X1$ 和 $U3D=X1-X0$ 。--skip-calibration 复用相机参数; --solver mesh --element all 运行网格分支。

多目 3D

```
python examples\multiview_3d.py --paths-config config\case_paths.yaml --config config\multiview_3d.yaml --solver subse
```

流程依次执行自标定、尺度恢复、相机对选择、ROI、两两二维 DIC、两两三维重建和曲面拼接。--solver 可选 subset|mesh|both, --resume 复用完整的两两二维场。

新手操作清单

新电脑或新案例请严格按以下顺序操作。CMake 编译成功不代表 Python 一定能加载同一个绑定模块，因此不要跳过导入检查。

- 克隆仓库，创建并激活唯一的 Python/Conda 环境。
- 在该环境安装 Python 与 C++ 依赖。
- 以 TRADITIONAL_DIC_BUILD_PYTHON=ON 配置并编译。
- 在仓库根目录执行手册中的 import traditional_dic 检查。
- 复制 config/case_paths.yaml 为新案例路径 YAML，例如 config/my_case_paths.yaml；不要把案例路径写入算法 YAML。
- 运行前确认图像排序。程序按排序推断参考图、变形图和 ROI 的角色。
- 先使用默认参数跑一张单目 Subset 图，检查 stats.json、位移 CSV、叠加图和 strain.csv。
- 再逐项修改搜索范围、子区半径、形函数、网格尺寸、应变半径和质量阈值。
- 只有二维场趋势合理、标定板尺寸与图像无误后，才运行双目或多目三维链路。

最小单目案例

目录中至少放三张图，例如 001.bmp、002.bmp、003.bmp。按排序，001.bmp 是参考图，002.bmp 是变形图，003.bmp 是二值 ROI；ROI 非零像素为有效区域。时序案例将所有变形图放在参考与 ROI 之间，例如 001.bmp, 002.bmp, ..., 100.bmp, 101_roi.bmp。

设置 mono_2d.case_root 与 images_dir 后运行 Subset。每个中间文件都应生成独立结果目录。若 ROI 排在中间，程序会误将它当成变形图；应通过重命名或调整目录解决。

最小双目案例

案例根目录必须包含左右图像目录、ROI 图和左右棋盘格目录。左右目录首图是无载参考双目图，末图是变形双目图。左右标定目录必须有相同数量的对应棋盘格图。board.rows/cols 是内角点数，spacing 是目标世界单位下的物理间距。

配置参数详解

Subset 参数

subset.radius

是相关子区像素半径，应大到能包含充分散斑、小到不跨越过大的位移梯度。shape_function.order 选择一阶仿射或二阶形函数。optimization.method 为 icgn 或 forward_gauss_newton，correlation.criterion 为 znssd 或 ssd。integer_search.search_radius 必须覆盖预期整数位移。reliability_propagation.spacing 只控制求解格点，不是应变参数。

strain.radius 与子区半径独立，是虚拟应变计半径。增大可降低噪声，减小可保留局部化。min_samples 防止边界邻域拟合不稳。大变形使用 green_lagrange；小位移小转动时才使用 infinitesimal。

Mesh 参数

mesh_generation.target_element_size

决定网格密度；单元越小，空间分辨率与计算时间越高。regularization_alpha 平滑位移梯度，但过大可能抹掉真实局部特征。FFT 初始化的 window_size、search_radius

必须匹配散斑与位移量。Mesh 应变使用节点连接关系，因此没有 spacing 或 radius；需通过网格质量、图像质量和正则化改善应变质量。

三维参数

双目的 workflow.calibrate/compute_fields/reconstruct 控制阶段开关。max_znssd、min_correlation、max_reprojection_error_px 控制点剔除。多目的 camera_pair_selection 控制相机对，maskGen 控制特征 ROI，scale 必须与棋盘格物理尺寸一致。保持 strain.enabled: true 才会输出有效三角面的应变。

可复现实验流程

每次报告的结果应保存输入图、算法 YAML、路径 YAML、Git commit 和运行 JSON。实际执行顺序可为：

```
git rev-parse HEAD
python examples\subset_2d.py --paths-config config\my_case_paths.yaml --config config\subset_2d.yaml
python examples\stereo_3d.py --paths-config config\my_case_paths.yaml --stereo-config config\stereo_3d.yaml --calibrat
python examples\multiview_3d.py --paths-config config\my_case_paths.yaml --config config\multiview_3d.yaml --solver su
```

验收时检查：图像角色、ROI 覆盖图、有效点比例、位移方向/量级、ZNSSD 和重投影误差分布、刚体运动下的应变、尺度恢复后的物理单位。图像看起来平滑并不等于标定或相关正确。

应变与结果

Subset 的 strain.radius 是 Ncorr 风格虚拟应变计半径：在圆邻域内对 u, v 作一阶最小二乘平面拟合，输出位移梯度及 exx, eyy, exy 。Mesh 使用单元连接得到的一环节点最小二乘拟合，不设置 subset 的 spacing 或 radius。measure: green_lagrange 使用有限 Green-Lagrange 应变，infinitesimal 使用对称位移梯度。

三维应变采用逐三角 Cosserat 有限应变：由三角面参考/变形顶点构造 F ，计算 $C=F^ATF$ 、 $B=FF^T$ 、Green-Lagrange、Euler-Almansi、主应变、等效应变、剪应变和面积变化，输出 stereo_3d_strain_faces.csv。

Python 接口

所有数组均为 NumPy；二维点/位移为 $N \times 2$ ，三维点为 $N \times 3$ ，Python 元素索引从 0 开始。

根包与二维求解

接口	参数	返回
subset(reference, deformed, config=None, roi=None, **overrides)	两张灰度图，YAML 映射/路径，二值 ROI，半径/搜索/种子/spacing/迭代覆盖项	$x, y, u, v, du_dx, du_dy, dv_dx, dv_dy, correlation, valid$ 字典
mesh(reference, deformed, nodes, elements, element_type, config=None, **overrides)	图像、 $N \times 2$ 节点、连接、T3/Q4/Q8、配置	节点场字典
generate_mesh_from_roi(roi, config=None)	ROI、网格配置	网格数据
generate_annulus_meshes_from_mask(mask, config=None)	环形 ROI、配置	T3/Q4/Q8 网格集合
reconstruct_from_fields(...)/reconstruct_from_field_files(...)	三个双目场或 CSV 目录、左右相机、质量参数	三维重建结果及文件

配置、核心、后处理

接口	参数	返回
<code>load_config(path)</code>	YAML 路径	字典
<code>normalize_subset_config(config)/normalize_mesh_config(config)</code>	字典或 None	后端可用配置
<code>core.load_image(path)/core.load_mask(path)</code>	图像/掩膜路径	后端对象
<code>core.normalize_image(image, method='none')</code>	图像、归一化方法	图像
<code>postprocess.save_least_squares_strain_csv(path, points, displacement, radius=None, elements=None, min_samples=6, green_lagrange=True)</code>	点、位移；Subset 给 radius, Mesh 给 elements	写应变 CSV
<code>compute_least_squares_strain_2d(points, displacement, radius, min_samples=6, green_lagrange=True)</code>	Subset 场	最小二乘应变记录
<code>compute_mesh_least_squares_strain_2d(nodes, displacement, elements, min_samples=3, green_lagrange=True)</code>	Mesh 场	最小二乘应变记录
<code>compute_surface_strain(faces, points_ref, points_def, valid_faces)</code>	三角面、参考/变形三维点、有效面	三角面应变记录

后三个底层计算接口位于 `_traditional_dic.postprocess`。

标定接口

`calibration.make_board`、`make_detection_options`、`make_mono_options`、`make_stereo_options`、`make_self_calibration_options`、`make_scale_options` 均接收 YAML 路径或字典并生成后端对象。`detect_calibration_board(image_path, board=None, config=None, options=None, return_raw=False)` 检测标定板。`calibrate_mono_zhang(image_paths, board, options=None)`、`calibrate_stereo_zhang(left_paths, right_paths, board, options=None)`、`calibrate_multiview_colmap_like(image_paths, config=None, options=None, return_raw=False)` 返回标定字典。`calibrate_*_from_points` 接收图像点/世界点；`estimate_multiview_chessboard_scale` 求尺度；`*_to_dict` 与 `save_json(data, path)` 用于序列化。

多目接口

接口	主要参数	返回
<code>select_camera_pairs(calibration, options=None)</code>	标定和 <code>CameraPairSelectionOptions</code>	相机对结果
<code>generate_pair_masks_from_calibration(case_root, calibration, config=None, pair_selection=None, output_dir=None, options=None)</code>	案例、标定、YAML/选项	ROI 结果
<code>generate_masks_from_calibration(...)/build_masks_from_calibration(...)</code>	标定、图像尺寸/图像、mask 选项	相机 mask
<code>compute_pairwise_2d_dic(case_root, calibration=None, config=None, pair_selection=None, subset_config=None, mesh_config=None, output_dir=None, options=None)</code>	案例、标定、配置、 <code>PairwiseDICO</code> <code>ptions</code>	两两二维结果
<code>compute_pairwise_3d_dic(case_root, calibration=None, config=None, pair_selection=None, field_dir=None, output_dir=None, options=None)</code>	两维场、 <code>Pairwise3DOptions</code>	两两三维结果
<code>recover_multiview_calibration_scale(case_root, calibration=None, config=None, options=None)</code>	标定、尺度选项	尺度结果
<code>stitch_pairwise_3d_surfaces(case_root, config=None, pair_selection=None, pairwise_3d_dir=None, output_dir=None, options=None)</code>	两两曲面、拼接选项	拼接结果
<code>save_pair_selection_report(result, path)</code>	相机对结果、JSON 路径	无
<code>multiview(reference_images, deformed_images=None, calibration=None, solver='subset')</code>	兼容快捷输入	workflow 结果

选项数据类为 `MultiviewMaskOptions`、`CameraPairSelectionOptions`、`PairwiseDICOOptions`、`Pairwise3DOptions`、`MultiviewScaleRecoveryOptions`、`PairwiseSurfaceStitchOptions`；可传数据类或只含需覆盖字段的字典。

可视化与曲面拼接

`visualization_dir_for_result(case_root,result_path,result_root='result')` 映射可视化目录。`densify_2d_mesh_displacement_field(nodes,elements,u,v,element_type,valid=None,samples_per_axis=17)` 生成稠密场。`plot_2d_field_overlay`、`plot_3d_scatter_field`、`plot_3d_surface_field`、`plot_stitched_surface_fields`

写图。`surface_stitching.stitch_surfaces(meshes,min_gap_factor=0.2)` 合并

`SurfaceMesh`；`clean_stitched_surface(...)` 清除离群；`write_stitch_visualizations(...)` 输出图像。

常见问题

- `_traditional_dic` 导入失败：以 `TRADITIONAL_DIC_BUILD_PYTHON=ON` 重新构建，并确认 `python/` 位于 `PYTHONPATH`。
- SIFT/标定不可用：在当前环境安装 OpenCV 后重新配置 CMake。
- 场无效：检查图像排序、ROI、相关阈值、搜索半径和初始化。
- 应变噪声大：增加 Subset `strain.radius`；Mesh 应改善网格质量或正则化，不能把 `subset spacing` 用作 Mesh 应变参数。
- 多目尺度失败：检查棋盘格、尺寸单位、相机重叠和标定质量。